

Homework 1 - Percolation

Course: CIS 387 - Analysis of Algorithms - Fall 2026

Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: September 14 at 11:59pm

Complete all the work in this folder, then zip the folder and upload to Canvas.

You must put your name on each java file you edit in the submission

Note: You may not use any AI assistance on this assignment.

For 2 extra credit points, answer the following questions (using the text box on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?

Your responses will be helpful feedback for refining these assignments in future courses.

Grading Information

The homework will be graded according to this rubric. Note that the bulk of credit is from parts 1 and 2. *If you can't complete everything, it is better to submit partial work than miss the deadline.*

Points	Description
20	Effort: Does the program compile and run? Was a reasonable attempt made?
10	readGridFromFile: Does the method create the correct array when reading a file?
10	printGrid: Does the method print the grid data in the right format?
30	percolates: Does the method correctly determine whether a grid percolates using the union find algorithm?
10	createRandomGrid: Does the method create a random grid using the given N and p?
10	DoublingTest: Does the program correctly run the doubling test as outlined in part 4?
10	Comments: Did you write comments in ProbabilityTest.java and DoublingTest.java that answer the questions in parts 3 and 4?

Setup

All the code and files needed for this homework are available on the course git repository:

<https://github.com/amininger-mu/CIS387F26Code>

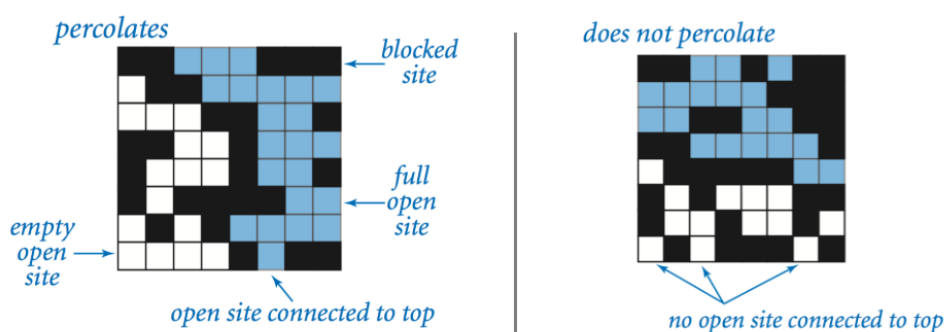
Percolation - Overview

Percolation is the process by which water or other liquid permeates through a layer of material. Think of water travelling through coffee grounds in a pour-over or rain runoff seeping into the ground.

We will create a model of a percolating system and apply the Union-Find algorithm to determine whether it can percolate or not.

Consider a $N \times N$ grid of squares, or **sites**. Each site can be either **open** or **blocked**. Water can move between open sites in the 4 cardinal directions (up/down/left/right).

We want to answer the following question: Given a $N \times N$ grid of sites, with each site having an independent probability p of being open, what is the probability that the grid *percolates*: meaning there is a path for water to travel from the top row to the bottom row?



In the above examples, the left one does percolate, because there is a path of open sites that leads from the top to the bottom. The right does not percolate.

1. Problem Setup

We will start with a representation of a percolation problem, and a file format to store it.

We will represent the grid as a 2D array of booleans (size $N \times N$). The value at `arr[i][j]` will be true if the site at row i , col j is open, and false if it is closed. A site's *neighbors* will be the (up to) 4 sites to the north, south, east, or west. The challenge is to determine whether a given grid *percolates*, meaning there is an open path from an open site in the top row to an open site in the bottom row.

To make testing easier, I have defined the following data format:

- The first line contains a single integer N , which will be the width/height of the grid
- The next N lines will each be of N characters, where a space represents open (true), and an `X` represents closed (false).

See `small-test.txt`:

```
5
X XX
  XX
XX X
XX X
X X X
```

Open the file called `Percolation.java`, and implement the following methods:

public static boolean[][] readGridFromFile(String filename)

Reads the given file (in the format described above) and uses it to create and initialize a 2D array of booleans.

public static void printGrid(boolean[][] grid)

Prints the given grid to the screen using the same format as the file (print the number N , then N lines of X's or spaces)

An example for printing a 4×4 grid might look like:

```
4
X XX
X X
  X
XX X
```

2. Translating the Problem

Once we have our grid of booleans describing a percolating system, we need to determine whether or not it actually percolates. Implement this method:

```
public static boolean percolates(boolean[][] grid)
```

You will need to figure out how to map the percolation problem onto an instance of the union find algorithm in order to complete this method. To do so, you will need to answer these questions:

- How will you map sites in the grid onto sites in the union find structure?
- Which sites will you connect (union)?
- How will you determine whether any site in the top row is connected to any site in the bottom?

Testing

To test, you can run the main program in **Percolation.java**. This expects 1 filename as an argument, and then loads the grid, calls **percolates**, and prints the result. I have provided a few example grid files (**small-test.txt**, **blocked.txt**, **medium-test.txt**), but I recommend you make your own.

3. Estimating Probability of Percolation

We will use your solution to answer the following question: What is the probability a random grid percolates? How does this depend on the probability $p = P(\text{open})$?

It is very difficult to calculate this exact probability analytically. Instead we will use *Monte Carlo Simulation* to run a set of random trials to estimate the probability.

First, we need a way to create a random grid. Implement the following method in **Percolation.java**:

```
public static boolean[][] createRandomGrid(int n, double p)
```

Create and returns a 2D array of booleans of size $N \times N$, where each **arr[i][j]** in the returned grid indicates whether the site at row i , col j is open. Randomly initialize the grid so that each site is open (true) with probability p .

ProbabilityTest.java

Once you have implemented the **createRandomGrid** method, look at the **ProbabilityTest.java** program. This will set the $P(\text{open})$ site probability p to 0.2, then run 100 trials of creating random grids and determining whether they percolate. It then prints the fraction that succeeded and increases the $P(\text{open})$ number.

Run this **ProbabilityTest.java** and look at the results. **Write a comment in ProbabilityTest.java** describing what you observe in the data. How does the choice of $p = P(\text{open})$ affect the probability a system percolates? What value of p seems to result in a 50/50 chance of it percolating?

Note! To make this efficient, make sure you are using the **WeightedQuickUnionUF** algorithm in your **percolates** method.

4. Measuring Performance

DoublingTest.java

We will now compare the performance of our three union find algorithms on this problem.

To do this, implement the **DoublingTest.java** main method in the following way:

1. Start with $N = 5$, create a random $N \times N$ grid using probability $p=0.55$.
2. Measure the time it takes to call the 'percolates' function from part 2.
3. Repeat this for a total of 3 trials, and print the average time (along with N).
4. Multiply N by 2, and repeat steps 1-3.
5. Stop when the average time exceeds 1 second.

Run this 3 times, once for each of the 3 algorithms: **QuickFindUF**, **QuickUnionUF**, and **WeightedQuickUnionUF** (change the type of algorithm in the **percolates** method, making sure to recompile).

In DoublingTest.java, record your results in a comment